

AML MINIPROJECT

TEAM MEMBERS: Rajasi Barapatre & Ketaki Paranjape

UCE NUMBERS: UCE2021507 & UCE2021547

BATCH: B4

DATASET:

<https://www.kaggle.com/datasets/cid007/mental-disorder-classification>

CODE:

```
!pip install catboost
import pandas as pd
from sklearn.model_selection import train_test_split
from catboost import CatBoostClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accuracy_score
import lightgbm as lgb
import xgboost as xgb
from sklearn.ensemble import AdaBoostClassifier
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
from sklearn.svm import SVC
```

```
data = pd.read_csv('Dataset-Mental-Disorders.csv')
data.head()
```

	Patient Number	Sadness	Euphoric	Exhausted	Sleep disorder	Mood Swing	Suicidal thoughts	Anorxia	Authority Respect	Try- Explanation	Aggressive Response	Stress & Move- On	Nervous Break- down	Admit Mistakes	Overthinking
0	Patient-01	Usually	Seldom	Sometimes	Sometimes	YES	YES	NO	NO	YES	NO	NO	YES	YES	YES
1	Patient-02	Usually	Seldom	Usually	Sometimes	NO	YES	NO	NO	NO	NO	NO	NO	NO	NO
2	Patient-03	Sometimes	Most- Often	Sometimes	Sometimes	YES	NO	NO	NO	YES	YES	NO	YES	YES	NO
3	Patient-04	Usually	Seldom	Usually	Most-Often	YES	YES	YES	NO	YES	NO	NO	NO	NO	NO
4	Patient-05	Usually	Usually	Sometimes	Sometimes	NO	NO	NO	NO	NO	NO	NO	YES	YES	YES

```
data.columns
```

```
Index(['Patient Number', 'Sadness', 'Euphoric', 'Exhausted', 'Sleep disorder',  
      'Mood Swing', 'Suicidal thoughts', 'Anorxia', 'Authority Respect',  
      'Try-Explanation', 'Aggressive Response', 'Ignore & Move-On',  
      'Nervous Break-down', 'Admit Mistakes', 'Overthinking',  
      'Sexual Activity', 'Concentration', 'Optimism', 'Expert Diagnose'],  
      dtype='object')
```

```
data.shape
```

```
(120, 19)
```

```
data.isnull().sum()
```

```
Patient Number    0  
Sadness           0  
Euphoric          0  
Exhausted         0  
Sleep disorder    0  
Mood Swing        0  
Suicidal thoughts 0  
Anorxia           0  
Authority Respect  0  
Try-Explanation   0  
Aggressive Response 0  
Ignore & Move-On   0  
Nervous Break-down 0  
Admit Mistakes    0  
Overthinking      0  
Sexual Activity    0  
Concentration      0  
Optimism           0  
Expert Diagnose    0  
dtype: int64
```

```
data.columns
```

```
Index(['Patient Number', 'Sadness', 'Euphoric', 'Exhausted', 'Sleep disorder',  
      'Mood Swing', 'Suicidal thoughts', 'Anorxia', 'Authority Respect',  
      'Try-Explanation', 'Aggressive Response', 'Ignore & Move-On',  
      'Nervous Break-down', 'Admit Mistakes', 'Overthinking',  
      'Sexual Activity', 'Concentration', 'Optimism', 'Expert Diagnose'],  
      dtype='object')
```

```
data = data.drop(columns=['Patient Number'])
```

```
data['Concentration'] =
```

```
data['Concentration'].astype(str).str.extract(r'(\d+)').astype(float).astype('Int64')
```

```
data['Optimism'] = data['Optimism'].astype(str).str.extract(r'(\d+)').astype(float).astype('Int64')
data['Sexual Activity'] = data['Sexual
Activity'].astype(str).str.extract(r'(\d+)').astype(float).astype('Int64')
print(data)
```

	Sadness	Euphoric	Exhausted	Sleep disorder	Mood Swing	\
0	Usually	Seldom	Sometimes	Sometimes	YES	
1	Usually	Seldom	Usually	Sometimes	NO	
2	Sometimes	Most-Often	Sometimes	Sometimes	YES	
3	Usually	Seldom	Usually	Most-Often	YES	
4	Usually	Usually	Sometimes	Sometimes	NO	
..	...	...	...	...	...	
115	Most-Often	Seldom	Usually	Sometimes	NO	
116	Sometimes	Sometimes	Sometimes	Seldom	YES	
117	Usually	Sometimes	Usually	Sometimes	YES	
118	Usually	Sometimes	Seldom	Seldom	NO	
119	Sometimes	Usually	Seldom	Usually	NO	

	Suicidal thoughts	Anorxia	Authority	Respect	Try-Explanation	\
0	YES	NO	NO	YES		
1	YES	NO	NO	NO		
2	NO	NO	NO	YES		
3	YES	YES	NO	YES		
4	NO	NO	NO	NO		
..	...	...	...	...		
115	YES	NO	NO	YES		
116	NO	NO	NO	NO		
117	NO	YES	YES	NO		
118	YES	YES	NO	YES		
119	NO	NO	YES	YES		

	Aggressive Response	Ignore & Move-On	Nervous Break-down	Admit Mistakes	\
0	NO	NO	YES	YES	
1	NO	NO	NO	NO	
2	YES	NO	YES	YES	
3	NO	NO	NO	NO	
4	NO	NO	YES	YES	
..	...	...	...	...	
115	NO	YES	NO	NO	
116	YES	NO	NO	NO	
117	NO	NO	YES	NO	
118	YES	YES	NO	YES	
119	NO	YES	YES	NO	

	Overthinking	Sexual Activity	Concentration	Optimism	Expert Diagnose
0	YES	3	3	4	Bipolar Type-2
1	NO	4	2	5	Depression
2	NO	6	5	7	Bipolar Type-1
3	NO	3	2	2	Bipolar Type-2
4	YES	5	5	6	Normal
..	...	...	...	...	...
115	YES	2	5	3	Depression
116	YES	6	7	8	Bipolar Type-1
117	YES	1	5	3	Bipolar Type-2
118	YES	7	7	7	Depression
119	NO	7	3	8	Normal

[120 rows x 18 columns]

```

data= data.rename(columns={'Optimism': 'Optimism'})
import seaborn as sns
import matplotlib.pyplot as plt
label_encoder = LabelEncoder()

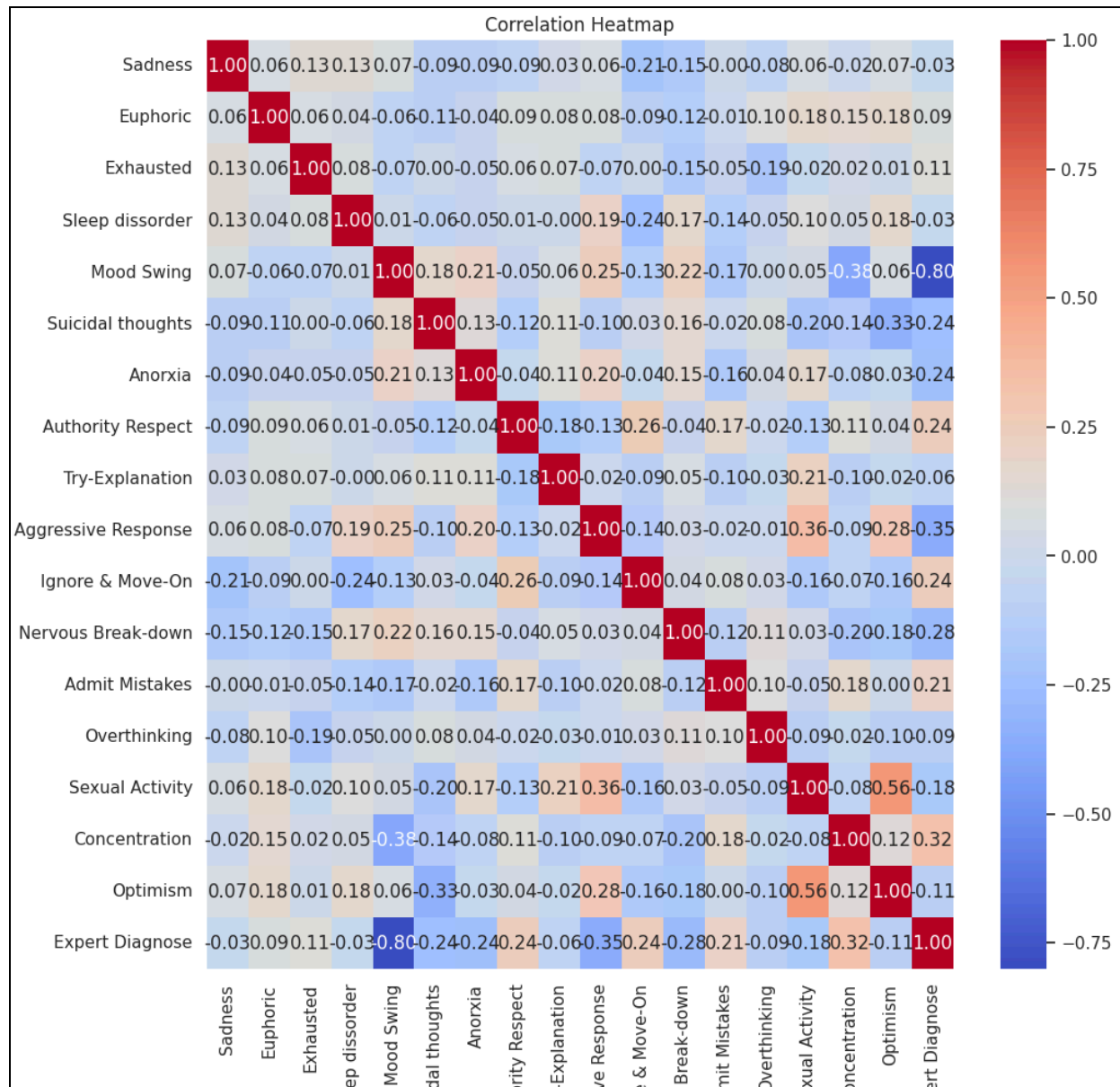
# Iterate over each categorical column
for col in data.columns:
    if data[col].dtype == 'object':
        data[col] = label_encoder.fit_transform(data[col])

sns.set(style="whitegrid")
correlation_matrix = data.corr()

plt.figure(figsize=(11, 11))
heatmap = sns.heatmap(correlation_matrix, annot = True, fmt = ".2f", cmap = 'coolwarm')

plt.title('Correlation Heatmap')
plt.show()

```



CATBOOST CLASSIFIER

```
X = data.drop('Expert Diagnose', axis=1)
```

```
y = data['Expert Diagnose']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
clf = CatBoostClassifier(random_state=42, verbose=0)
```

```
clf.fit(X_train, y_train, cat_features=['Sadness', 'Euphoric', 'Exhausted', 'Sleep disorder', 'Mood Swing',
```

```
'Suicidal thoughts', 'Anorxia', 'Authority Respect', 'Try-Explanation',
```

```
'Aggressive Response', 'Ignore & Move-On', 'Nervous Break-down',  
'Admit Mistakes', 'Overthinking', 'Sexual Activity', 'Concentration',  
'Optimism']])
```

```
y_pred = clf.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)  
report = classification_report(y_test, y_pred)
```

```
print(f'Accuracy: {accuracy}')  
print(f'Classification Report:\n{report}')
```

```
Accuracy: 0.8333333333333334
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	1.00	0.50	0.67	4
1	0.83	1.00	0.91	5
2	0.80	1.00	0.89	8
3	0.83	0.71	0.77	7
accuracy			0.83	24
macro avg	0.87	0.80	0.81	24
weighted avg	0.85	0.83	0.82	24

VOTING CLASSIFIER (HARD)

```
encoder = LabelEncoder()
```

```
for col in data.columns:  
    if col != 'Expert Diagnose':  
        data[col] = encoder.fit_transform(data[col])  
X = data.drop('Expert Diagnose', axis=1)  
y = data['Expert Diagnose']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
rf_clf = RandomForestClassifier(random_state=42)  
dt_clf = DecisionTreeClassifier(random_state=42)  
knn_clf = KNeighborsClassifier(n_neighbors=9)
```

```
voting_clf = VotingClassifier(  
    estimators=[('rf', rf_clf), ('dt', dt_clf), ('knn', knn_clf)],  
    voting='hard'  
)
```

```
voting_clf.fit(X_train, y_train)
```

```
y_pred = voting_clf.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
report = classification_report(y_test, y_pred)
```

```
print(f'Accuracy: {accuracy}')
```

```
print(f'Classification Report:\n{report}')
```

Accuracy: 0.75

Classification Report:

	precision	recall	f1-score	support
0	0.50	0.75	0.60	4
1	1.00	0.80	0.89	5
2	0.78	0.88	0.82	8
3	0.80	0.57	0.67	7
accuracy			0.75	24
macro avg	0.77	0.75	0.74	24
weighted avg	0.78	0.75	0.75	24

VOTING CLASSIFIER (SOFT)

```
voting_clf1 = VotingClassifier(  
    estimators=[('rf', rf_clf), ('dt', dt_clf), ('knn', knn_clf)],  
    voting='soft'  
)
```

```
voting_clf1.fit(X_train, y_train)
```

```
y_pred = voting_clf1.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
report = classification_report(y_test, y_pred)
```

```
print(f'Accuracy: {accuracy}')
```

```
print(f'Classification Report:\n{report}')
```

Accuracy: 0.75

Classification Report:

	precision	recall	f1-score	support
0	0.50	0.75	0.60	4
1	1.00	0.80	0.89	5

2	0.78	0.88	0.82	8
3	0.80	0.57	0.67	7

accuracy		0.75	24
macro avg	0.77	0.75	0.74
weighted avg	0.78	0.75	0.75

LIGHT GBM

```

label_encoder = LabelEncoder()
data['Expert Diagnose'] = label_encoder.fit_transform(data['Expert Diagnose'])

X = data.drop('Expert Diagnose', axis=1)
y = data['Expert Diagnose']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Convert the dataset into LightGBM format
train_data = lgb.Dataset(X_train, label=y_train)
test_data = lgb.Dataset(X_test, label=y_test)

# Set the hyperparameters for LightGBM
params = {
    'objective': 'multiclass', # Multiclass classification
    'num_class': len(y.unique()), # Number of classes in the target variable
    'metric': 'multi_logloss', # Evaluation metric
    'num_leaves': 31, # Maximum number of leaves in one tree
    'learning_rate': 0.05, # Learning rate
    'feature_fraction': 0.9, # Fraction of features to use in each iteration
    'bagging_fraction': 0.8, # Fraction of data to use in each iteration (bagging)
    'bagging_freq': 5, # Frequency for bagging
    'verbose': 0 # Suppress output
}

num_round = 100 # Number of boosting rounds
bst = lgb.train(params, train_data, num_round, valid_sets=[test_data])

y_pred = bst.predict(X_test, num_iteration=bst.best_iteration)
y_pred = [list(x).index(max(x)) for x in y_pred] # Convert predicted probabilities to class labels

# Decode the numerical labels back to original class labels
y_test = label_encoder.inverse_transform(y_test)
y_pred = label_encoder.inverse_transform(y_pred)

```



```
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
```

```
print(f'Accuracy: {accuracy}')
print(f'Classification Report:\n{report}')
```

Accuracy: 0.7916666666666666

Classification Report:

	precision	recall	f1-score	support
0	0.75	0.75	0.75	4
1	1.00	0.80	0.89	5
2	0.73	1.00	0.84	8
3	0.80	0.57	0.67	7
accuracy			0.79	24
macro avg	0.82	0.78	0.79	24
weighted avg	0.81	0.79	0.79	24

XG BOOST

```
label_encoder = LabelEncoder()
data['Expert Diagnose'] = label_encoder.fit_transform(data['Expert Diagnose'])
```

```
X = data.drop('Expert Diagnose', axis=1)
y = data['Expert Diagnose']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
xgb_clf = xgb.XGBClassifier(objective='multi:softmax', num_class=len(y.unique()))
xgb_clf.fit(X_train, y_train)
y_pred = xgb_clf.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
```

```
print(f'Accuracy: {accuracy}')
print(f'Classification Report:\n{report}')
```

Accuracy: 0.75

Classification Report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.60	0.75	0.67	4
1	1.00	0.80	0.89	5
2	0.78	0.88	0.82	8
3	0.67	0.57	0.62	7

accuracy			0.75	24
macro avg	0.76	0.75	0.75	24
weighted avg	0.76	0.75	0.75	24

ADABOOST METHOD

```
ada_clf = AdaBoostClassifier()
ada_clf.fit(X_train, y_train)
y_pred = ada_clf.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
```

```
print(f'Accuracy: {accuracy}')
print(f'Classification Report:\n{report}')
```

Accuracy: 0.625

Classification Report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	1.00	0.50	0.67	4
1	0.80	0.80	0.80	5
2	0.62	0.62	0.62	8
3	0.44	0.57	0.50	7

accuracy			0.62	24
macro avg	0.72	0.62	0.65	24
weighted avg	0.67	0.62	0.63	24

NEURAL NETWORK

```
X_train = np.array(X_train)
X_test = np.array(X_test)
y_train = np.array(y_train)
y_test = np.array(y_test)
```

```
model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(16, activation='relu'),
```

```

Dense(len(np.unique(y_train)), activation='softmax')
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=0)
y_pred = np.argmax(model.predict(X_test), axis=-1)

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f'Accuracy: {accuracy}')
print(f'Classification Report:\n{report}')

```

1/1 [=====] - 0s 87ms/step

Accuracy: 0.6666666666666666

Classification Report:

	precision	recall	f1-score	support
0	0.60	0.75	0.67	4
1	0.75	0.60	0.67	5
2	0.73	1.00	0.84	8
3	0.50	0.29	0.36	7
accuracy			0.67	24
macro avg	0.64	0.66	0.63	24
weighted avg	0.64	0.67	0.64	24

SVM

```

svm_clf = SVC(kernel='rbf', gamma='scale')
svm_clf.fit(X_train, y_train)
y_pred = svm_clf.predict(X_test)

```

```

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

```

```

print(f'Accuracy: {accuracy}')
print(f'Classification Report:\n{report}')

```

Accuracy: 0.7083333333333334

Classification Report:

	precision	recall	f1-score	support
0	0.67	1.00	0.80	4
1	0.50	0.80	0.62	5

2	1.00	0.62	0.77	8
3	0.80	0.57	0.67	7

accuracy			0.71	24
macro avg	0.74	0.75	0.71	24
weighted avg	0.78	0.71	0.71	24

STACKING

```

from sklearn.base import clone
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression

base_classifiers = [
    ('rf', RandomForestClassifier(random_state=42)),
    ('gb', GradientBoostingClassifier(random_state=42)),
    ('knn', KNeighborsClassifier())
]

meta_classifier = LogisticRegression()

def stacking(base_classifiers, meta_classifier, X_train, y_train, X_test):
    meta_features_train = np.zeros((len(X_train), len(base_classifiers)))
    meta_features_test = np.zeros((len(X_test), len(base_classifiers)))

    for i, (name, clf) in enumerate(base_classifiers):
        clf.fit(X_train, y_train)
        meta_features_train[:, i] = clf.predict(X_train)
        meta_features_test[:, i] = clf.predict(X_test)

    meta_classifier.fit(meta_features_train, y_train)
    stacked_predictions = meta_classifier.predict(meta_features_test)

    return stacked_predictions

stacked_predictions = stacking(base_classifiers, meta_classifier, X_train, y_train, X_test)

accuracy = accuracy_score(y_test, stacked_predictions)
report = classification_report(y_test, stacked_predictions)

```

```
print(f'Stacked Model Accuracy: {accuracy}')
print(f'Classification Report:\n{report}')
```

Stacked Model Accuracy: 0.7083333333333334

Classification Report:

	precision	recall	f1-score	support
0	0.50	0.50	0.50	4
1	0.80	0.80	0.80	5
2	0.70	0.88	0.78	8
3	0.80	0.57	0.67	7
accuracy			0.71	24
macro avg	0.70	0.69	0.69	24
weighted avg	0.72	0.71	0.70	24

GAUSSIAN NAIVE BAYES

```
from sklearn.naive_bayes import GaussianNB
nb_clf = GaussianNB()
nb_clf.fit(X_train, y_train)
y_pred = nb_clf.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
```

```
print(f'Accuracy: {accuracy}')
print(f'Classification Report:\n{report}')
```

Accuracy: 0.5833333333333334

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.25	0.40	4
1	0.71	1.00	0.83	5
2	0.50	1.00	0.67	8
3	0.00	0.00	0.00	7
accuracy			0.58	24
macro avg	0.55	0.56	0.47	24
weighted avg	0.48	0.58	0.46	24

BERNOULLI NAIVE BAYES

```
from sklearn.naive_bayes import BernoulliNB
```

```

from sklearn.preprocessing import LabelEncoder, Binarizer
binarizer = Binarizer()
X_binarized = binarizer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_binarized, y, test_size=0.2,
random_state=42)

bnb_clf = BernoulliNB()
bnb_clf.fit(X_train, y_train)
y_pred = bnb_clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f'Accuracy: {accuracy}')
print(f'Classification Report:\n{report}')

```

Accuracy: 0.75

Classification Report:

	precision	recall	f1-score	support
0	0.60	0.75	0.67	4
1	0.83	1.00	0.91	5
2	0.70	0.88	0.78	8
3	1.00	0.43	0.60	7
accuracy			0.75	24
macro avg	0.78	0.76	0.74	24
weighted avg	0.80	0.75	0.73	24

KNN CLASSIFIER

```

knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_train)
y_pred = knn_clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f'Accuracy: {accuracy}')
print(f'Classification Report:\n{report}')

```

Accuracy: 0.5833333333333334

Classification Report:

	precision	recall	f1-score	support
0	0.25	0.25	0.25	4
1	0.67	0.40	0.50	5
2	0.58	0.88	0.70	8
3	0.80	0.57	0.67	7

accuracy			0.58	24
macro avg	0.57	0.52	0.53	24
weighted avg	0.61	0.58	0.57	24

DECISION TREE CLASSIFIER

```
dt_clf = DecisionTreeClassifier(random_state=0)
```

```
# Train the Decision Tree classifier
```

```
dt_clf.fit(X_train, y_train)
```

```
# Make predictions
```

```
y_pred = dt_clf.predict(X_test)
```

```
# Evaluate the model
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
report = classification_report(y_test, y_pred)
```

```
print(f'Accuracy: {accuracy}')
```

```
print(f'Classification Report:\n{report}')
```

Accuracy: 0.625

Classification Report:

	precision	recall	f1-score	support
0	0.33	0.25	0.29	4
1	0.67	0.80	0.73	5
2	0.67	0.75	0.71	8
3	0.67	0.57	0.62	7

accuracy			0.62	24
macro avg	0.58	0.59	0.58	24
weighted avg	0.61	0.62	0.61	24

RANDOM FOREST CLASSIFIER

```
from sklearn.model_selection import GridSearchCV

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)

# define the parameter grid
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

rf_classifier = RandomForestClassifier(random_state = 0)

# instantiate GridSearchCV
grid_search = GridSearchCV(estimator = rf_classifier, param_grid = param_grid, verbose = 0)

# fit the grid search to the data
grid_search.fit(X_train, y_train)

# set the best parameters
best_params = grid_search.best_params_
print("Best Parameters:", best_params)

Best Parameters: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2,
'n_estimators': 200}

optimized_rf_classifier = RandomForestClassifier(**best_params, random_state = 0) # dict
unpacking
optimized_rf_classifier.fit(X_train, y_train)

y_pred_test = optimized_rf_classifier.predict(X_test)
y_pred_train = optimized_rf_classifier.predict(X_train)

train_accuracy = accuracy_score(y_train, y_pred_train)
print(f"Training Accuracy: {train_accuracy:.2%}")

rf_accuracy = accuracy_score(y_test, y_pred_test)
print(f"Testing Accuracy: {rf_accuracy:.2%}")

print('Classification report:')
print(classification_report(y_test, y_pred_test))
```


Training Accuracy: 100.00%

Testing Accuracy: 87.50%

Classification report:

	precision	recall	f1-score	support
0	0.80	0.80	0.80	5
1	0.89	1.00	0.94	8
2	0.50	0.50	0.50	2
3	1.00	0.89	0.94	9
accuracy			0.88	24
macro avg	0.80	0.80	0.80	24
weighted avg	0.88	0.88	0.88	24